



ChainTrack

Blockchain Supply Chain Tracker

Detailed Implementation Plan

MVP-first • Local Hardhat → Sepolia • Fast light-themed UX



1. Executive Summary

ChainTrack is a blockchain-based supply-chain traceability system that gives each product a unique digital identity, records custody changes on-chain, and lets a customer verify a product through a QR code without logging in. The implementation follows a hybrid architecture: the smart contract is the source of truth for ownership, status, history and the product-data hash, while PostgreSQL stores descriptive information, search data and dashboard information.

The implementation is deliberately MVP-first. The core journey must work end-to-end on a local Hardhat chain before stretch features or Sepolia deployment are attempted.

Project goals

- Give every product a unique digital identity and QR code.
- Record custody changes and status updates as tamper-resistant blockchain history.
- Allow customers to verify authenticity and see the complete product journey without login.
- Enforce role and ownership rules in the smart contract, not only in the UI.
- Deliver a reliable, demo-ready system that works end to end.

Scope boundaries

In scope	Out of scope for MVP
Wallet login, roles, participant management	Real payments or cryptocurrency transfers
Product registration, hash, QR	Ethereum mainnet
Two-step custody transfer	IoT/GPS/temperature sensors
Location update and sale	Native mobile application
Public verification and timeline	Multi-tenant enterprise separation
Dashboards, search and filters	Legal/regulatory compliance features

Source basis: PRD goals, users, lifecycle and scope boundaries. ■filecite■turn7file0■L17-L54■

2. Target Architecture

ChainTrack is implemented as three primary layers with a blockchain indexer connecting on-chain events to PostgreSQL. Users send state-changing transactions directly through MetaMask; the backend never holds user private keys.

Layer	Technology	Primary responsibility
Smart contract	Solidity ^0.8.24 + Hardhat	Roles, ownership, statuses, transitions, history, hashes
Web app / API	Next.js App Router + TypeScript	UI, wallet interaction, API routes, auth, uploads, QR data
Database	PostgreSQL + Prisma	Off-chain metadata, profiles, indexed events, search, dashboard data
Blockchain client	ethers.js v6 + MetaMask	Wallet connection, reads, transaction submission, event reads
Validation	Zod	API/form input validation
QR	qrcode + html5-qrcode	QR generation and browser scanning
Testing	Hardhat + Chai, Vitest, Playwright	Contract, backend/utility and end-to-end verification

Architecture flow

Browser / Next.js UI → MetaMask → SupplyChain contract → blockchain
Browser → Next.js API → PostgreSQL
Indexer → reads contract events → mirrors searchable data into PostgreSQL

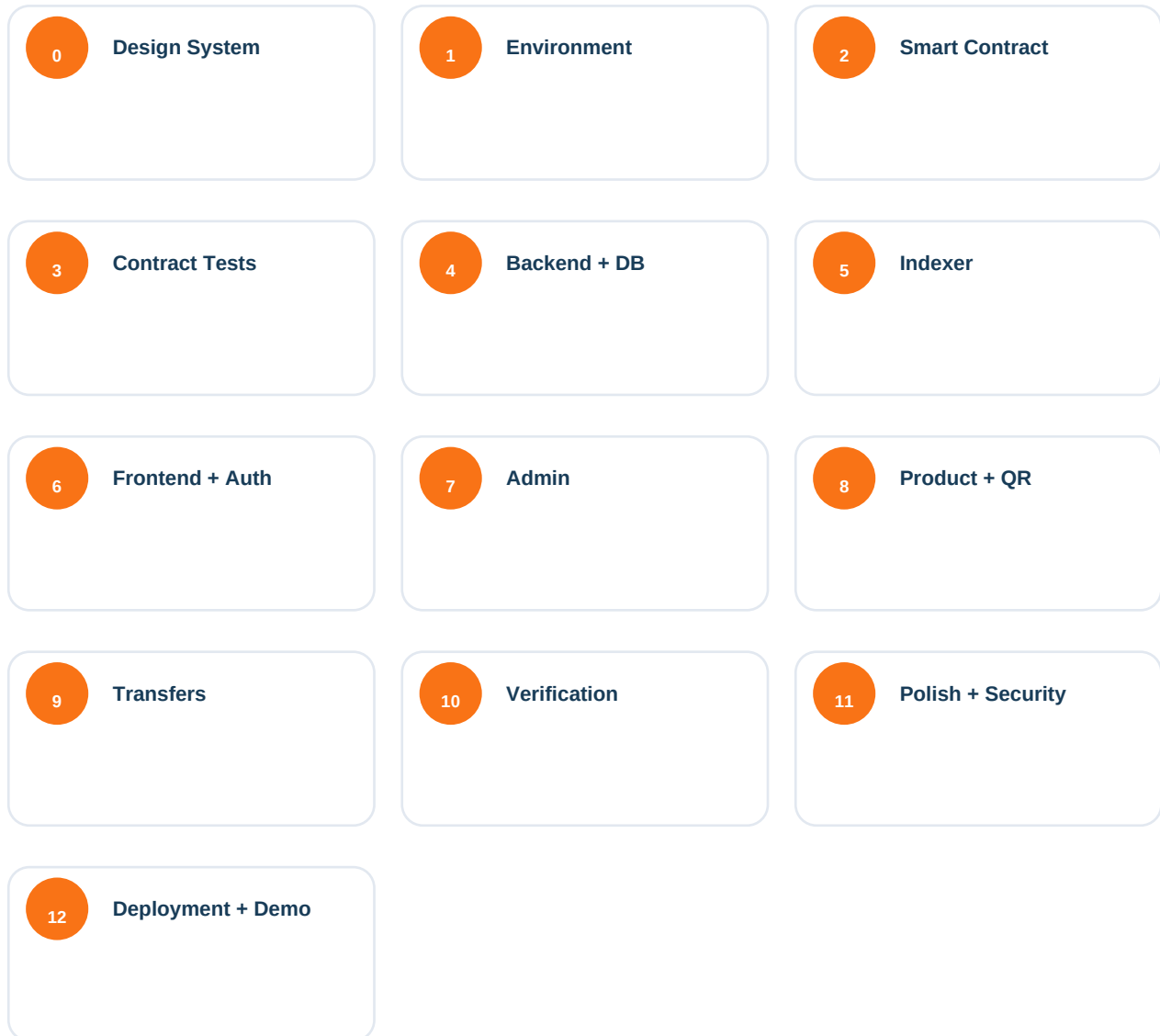
Source-of-truth rule

- Blockchain wins for product existence, owner, status, event history and data hash.
- Database wins for descriptive text, images and contact/profile details.
- The public verification page reads status and history directly from the blockchain so verification can still show chain data if the database is unavailable.

Source basis: TRD architecture and source-of-truth responsibilities. ■filecite■turn7file2■L176-L207■ ■filecite■turn7file6■L481-L494■

3. Implementation Roadmap

The project is divided into twelve implementation phases. Each phase has a clear gate. Do not move forward when the previous gate is unstable.



Phase	Primary output	Gate
0	Design system + screen blueprint	UI system approved
1	Local development environment	Hardhat + PostgreSQL + Next.js run
2	SupplyChain.sol	Contract compiles/deploys
3	Contract test suite	Valid/invalid transitions pass
4	PostgreSQL + API foundation	Migrations + API baseline work
5	Event indexer	Chain events mirror correctly
6	Wallet auth + app shell	Wallet → role dashboard

Phase	Primary output	Gate
7	Admin module	Participant lifecycle works
8	Product + QR	Product → hash → chain → QR
9	Transfer lifecycle	Manufacturer → Distributor → Retailer → Sold
10	Public verification	Authentic / Mismatch / Not Found
11	Quality hardening	Security + performance + E2E checks
12	Local → Sepolia demo	Repeatable final demo

4. Phase 0 — Design System & UI/UX Blueprint

Objective: finalize the visual and interaction system before feature implementation so later screens reuse the same components rather than being redesigned repeatedly.

Locked visual identity

Role	Color
Primary Navy	#173B57
Primary Orange	#F97316
Navy Light	#EAF1F5
Orange Light	#FFF1E8
Background	#F8F AFC
Surface	#FFFFFF
Text	#17202A
Muted	#64748B
Border	#E2E8F0
Verified / Sold	#16A34A
Pending / In Transit	#F59E0B
Error / Tampered	#DC2626

Component inventory

- App shell: sidebar, top bar, wallet connection, breadcrumbs and responsive navigation.
- Inputs: text, select, date, file upload, location, search and filters.
- Feedback: badges, toast, modal, confirmation, skeleton, empty state and error state.
- Blockchain UX: transaction progress modal, pending/success/failed states, transaction hash link.
- Supply-chain UX: product card, status badge, journey timeline, transfer card and QR card.
- Public verification: authenticity result, product summary, timeline and blockchain proof.

Animation rules

- Use purposeful motion only: state changes, feedback and navigation—not decorative motion everywhere.
- Prefer transform and opacity animations for smooth rendering.
- Target short transitions: roughly 150–200 ms for micro-interactions and 200–500 ms for larger UI transitions.
- Use a subtle orange motion cue for supply-chain movement and green only for verified/success states.
- Use skeleton loading rather than blank waiting screens.

Phase gate: all major components and screens are visually defined, responsive behavior is decided, and the Navy + Orange design system is frozen.

5. Phase 1 — Development Environment

Objective: create a reproducible local setup that can run the blockchain, database and web application together.

Repository

```
chaintrack/  
■■■ contracts/ # Hardhat project  
■ ■■■ contracts/ # SupplyChain.sol  
■ ■■■ test/ # contract tests  
■ ■■■ scripts/ # deploy.ts, seed.ts  
■■■ web/ # Next.js app  
■ ■■■ src/app/ # pages + API routes  
■ ■■■ src/lib/ # chain, hash, auth, db  
■ ■■■ src/components/  
■ ■■■ prisma/ # schema + migrations  
■■■ docker-compose.yml  
■■■ docs/  
■■■ README.md
```

Tasks

- Initialize Hardhat with TypeScript and Solidity ^0.8.24.
- Initialize Next.js App Router + TypeScript + Tailwind CSS.
- Add PostgreSQL through Docker Compose.
- Add Prisma and environment-variable handling with .env.example.
- Install ethers.js v6, MetaMask-compatible wallet flow, Zod and QR libraries.
- Create baseline scripts: chain, deploy:local, seed, dev and test.
- Create README with local prerequisites and setup steps.

Gate

A clean machine can start the local Hardhat node, PostgreSQL and Next.js application using the documented setup. The TRD targets a setup guide that can be followed in under 15 minutes.

Source basis: TRD repository, scripts and demo-readiness requirements. ■filecite■turn6file4■L329-L368■

6. Phase 2 — Smart Contract Core

Objective: implement the immutable business rules first. The UI must never be the only place enforcing permissions or state transitions.

Core data model

Type	Values / fields
Role	None, Manufacturer, Distributor, Retailer
Status	Created, InTransit, AtDistributor, AtRetailer, Sold
EventType	Registered, TransferInitiated, TransferAccepted, TransferRejected, LocationUpdate, Sold
Product	id, dataHash, manufacturer, currentOwner, pendingReceiver, status, createdAt
HistoryEntry	eventType, actor, counterparty, timestamp, location, note

Implementation order

- Participant registry: registerParticipant and setParticipantActive.
- Product registration: registerProduct with non-zero, unused dataHash and location validation.
- Transfer initiation: only current owner; only correct next role; product cannot already be InTransit.
- Transfer acceptance/rejection: only pending receiver; reject returns to the previous owner/status.
- Location updates: current owner only, and only in allowed holding states.
- Sale: retailer only, only AtRetailer; Sold is final.
- On-chain history and events for every write action.
- Custom errors and access modifiers for clear failure handling.

Contract deployment

Deploy SupplyChain, write deployment metadata and ABI to the web project, and record deployBlock so a fresh indexer knows where to begin.

Source basis: schema/contract design and deployment rules. [filecite turn7file3 L252-L309](#) [filecite turn7file8 L638-L646](#)

7. Phase 3 — Smart Contract Testing

Objective: prove the state machine independently of the frontend.

Required test groups

- Admin-only participant registration and activation/deactivation.
- Manufacturer-only product registration.
- Duplicate and empty hash rejection.
- Correct Manufacturer → Distributor transfer.
- Distributor → Retailer transfer.
- Wrong owner and wrong role rejection.
- Wrong receiver cannot accept/reject.
- Reject returns product to the previous state.
- Location update works only in valid holding states.
- Retailer can mark Sold; Sold blocks all future writes.
- Product-not-found behavior.
- Event emission and history order.

Coverage target

The TRD targets 100% of write functions covered and at least 90% line coverage for the smart contract suite.

Negative tests are especially important because the project's security model requires contract-level enforcement, not merely hidden UI actions.

Gate: all contract tests pass on a clean local chain before backend/frontend integration begins.

Source basis: TRD testing requirements and schema/contract test cases. ■filecite■turn7file7■L547-L568■
■filecite■turn7file8■L607-L636■

8. Phase 4 — PostgreSQL + Backend Foundation

Objective: build the off-chain data layer and API routes without duplicating blockchain business rules.

Core database entities

Entity	Purpose
Participant	Wallet, role, organization profile and active state
Product	Descriptive data, chainProductId, hash, confirmation state and current snapshot
ProductEvent	Indexed blockchain events for searchable lists
AuthNonce	Single-use wallet-login nonce with expiry
IndexerState	Last processed blockchain block

API responsibilities

- Wallet auth nonce issuance and signature verification.
- Protected participant/product/transfer queries.
- Product metadata and image storage.
- Dashboard statistics and paginated search.
- QR data generation.
- Verification helper logic and hash comparison.
- Zod validation on all incoming inputs.

Important rule: the backend may prepare metadata and read chain state, but user transactions are signed by the user's wallet.

Source basis: TRD architecture, responsibilities and security requirements. ■filecite■turn7file6■L481-L494■
■filecite■turn7file7■L524-L545■

9. Phase 5 — Blockchain Event Indexer

Objective: mirror blockchain events into PostgreSQL so dashboards, search and lists are fast, while keeping blockchain as the authoritative source.

Event mapping

Blockchain event	Database action
ParticipantRegistered	Upsert participant
ParticipantStatusChanged	Update participant active state
ProductRegistered	Insert event; link matching draft by dataHash
TransferInitiated	Insert ProductEvent
TransferAccepted	Insert ProductEvent
TransferRejected	Insert ProductEvent
LocationUpdated	Insert ProductEvent
ProductSold	Insert ProductEvent

Indexer rules

- Process events from deployBlock forward.
- Persist lastProcessedBlock so restarts continue safely.
- Use unique transaction/log identifiers so processing the same event twice is harmless.
- After a batch, snapshot affected products from getProduct at the relevant block to avoid reimplementing state transitions in the backend.
- Never delete blockchain-derived product/event history; participants are deactivated rather than deleted.

Gate: create a test that processes the same event batch twice and confirms the final database state is identical.

Source basis: schema/indexer rules and backend test requirements. ■filecite■turn7file4■L320-L353■ ■filecite■turn7file7■L561-L568■

10. Phase 6 — Frontend Shell + Wallet Authentication

Objective: create the responsive application shell and a secure wallet-based login flow.

Authentication flow

```
Connect Wallet
↓
Check chain/network
↓
Request one-time nonce
↓
Sign login message in MetaMask
↓
Verify signature server-side
↓
Read on-chain participant role
↓
Set httpOnly JWT session
↓
Redirect to role dashboard
```

Required UX states

- MetaMask not installed → clear install prompt.
- Wrong network → Switch Network action before signature.
- User rejects signature → no state change; clear cancellation message.
- Unregistered wallet → contact-admin message.
- Inactive participant → deactivated-account message.
- Pending transaction → disable duplicate submission and show confirmation progress.
- RPC failure → retry action and readable explanation.

Role navigation

Role	Primary screens
Admin	Dashboard, Participants, Products/Activity, Settings
Manufacturer	Dashboard, My Products, Create Product, Transfers, History
Distributor	Dashboard, Incoming, Inventory, Shipments, History
Retailer	Dashboard, Incoming, Inventory, Sold Products, History
Customer	Public Verify only; no login

Source basis: system login flow and PRD role model. ■filecite■turn7file1■L93-L145■ ■filecite■turn7file0■L46-L54■

11. Phase 7 — Admin & Participant Management

Objective: establish the trusted participant registry that controls who can operate the supply chain.

Admin workflow

```
Admin wallet
↓
Participants → Add participant
↓
Wallet + role + organization + contact/location
↓
MetaMask transaction
↓
Contract registry updated
↓
Indexer mirrors participant
↓
Participant can log in
```

Features

- Register Manufacturer, Distributor or Retailer.
- View wallet address, role, organization and active state.
- Activate/deactivate a participant.
- Prevent duplicate role assignment to the same wallet.
- Show clear transaction and failure states.
- Keep participant records rather than deleting them; deactivation is the lifecycle operation.

Gate: the demo admin can create one Manufacturer, one Distributor and one Retailer and each can authenticate with the correct role.

12. Phase 8 — Product Registration + QR

Objective: create the first complete manufacturer journey: product metadata → deterministic hash → blockchain registration → QR code.

Product registration pipeline

```
Form data
↓
Normalize canonical fields
↓
Compute product data hash
↓
Save metadata / draft off-chain
↓
Manufacturer signs registerProduct(hash, location)
↓
Contract creates Product ID
↓
Indexer confirms chain record
↓
Generate verification QR
```

Hash requirements

- Hash input must be canonical so equivalent data produces the same hash regardless of key order.
- The hash must include the product identity fields required by the project design, including the serial number refinement documented in the schema notes.
- The contract rejects a zero or already-used hash.
- Full descriptive data remains off-chain; the hash is the integrity anchor.

QR behavior

The QR should encode the public verification URL, for example `/verify/{productId}`. It must be downloadable/printable and readable from a phone camera.

Gate: manufacturer creates a product, sees a confirmed product ID, downloads its QR, and scanning the QR opens the correct public verification route.

Source basis: PRD product registration requirements and schema hash/indexer design. ■filecite■turn4file9■L689-L694■
■filecite■turn7file4■L357-L362■

13. Phase 9 — Supply Chain Transfer Lifecycle

Objective: implement the core business journey using a two-step handover model.

Lifecycle

```
Created
↓ initiate transfer
InTransit
↓ receiver accepts
AtDistributor
↓ initiate transfer
InTransit
↓ receiver accepts
AtRetailer
↓ retailer marks sold
Sold (FINAL)
```

Manufacturer → Distributor

- Manufacturer selects an active Distributor.
- Manufacturer supplies location and optional note.
- Transaction moves product to InTransit and records a history event.
- Only the named Distributor can accept or reject.
- Accept moves ownership to Distributor; reject returns to the previous state.

Distributor → Retailer

- Distributor can update location while holding the product.
- Distributor initiates transfer to an active Retailer.
- Retailer accepts or rejects.
- Acceptance moves ownership to Retailer.

Retail sale

- Only the Retailer can mark AtRetailer as Sold.
- Sold is a final state: all subsequent write attempts revert.

Gate: one product completes Manufacturer → Distributor → Retailer → Sold, including at least one rejection demo and one location update.

Source basis: PRD lifecycle and transfer rules. ■filecite■turn7file0■L56-L73■

14. Phase 10 — Public Verification & Authenticity

Objective: deliver the customer-facing feature that explains the value of blockchain without requiring the customer to understand wallets or Web3.

Verification pipeline

```
Scan QR / enter Product ID
↓
Read product state + history from blockchain
↓
Fetch descriptive metadata from database
↓
Recompute product hash
↓
Compare with on-chain dataHash
↓
Authentic / Data Mismatch / Not Found
```

Result states

Result	Condition	UI treatment
Authentic	Product exists on-chain and hash matches	Green verified result + complete timeline
Data Mismatch	Product exists but off-chain hash differs	Red/orange warning + integrity explanation
Not Found	Product ID does not exist on-chain	Clear not-found state + scan/search again

Verification page layout

- Hero authenticity result.
- Product identity: name, serial/product ID, category and image when available.
- Current status and current owner.
- Visual journey timeline: manufacturer → distributor → retailer → sold.
- Per-event timestamp, actor, counterparty, location and transaction hash.
- Expandable 'Blockchain Proof' section for technical evidence.
- Public page works without login and can still show chain data when PostgreSQL is unavailable.

Gate: the QR scan without login shows the correct authenticity result and a correctly ordered full timeline.

Source basis: PRD public verification requirements and TRD source-of-truth behavior. ■filecite■turn4file9■L706-L712■
■filecite■turn7file6■L491-L494■

15. Phase 11 — UX Polish, Security, Performance & Reliability

Objective: turn a technically working MVP into a reliable, professional demo without sacrificing rendering speed.

Security checklist

- No user private keys in frontend/backend.
- Every contract write protected by contract-level access rules.
- Zod validation on API/form inputs.
- Single-use, expiring auth nonces.
- httpOnly JWT session cookie.
- Rate limiting for authentication and public verification endpoints.
- File type/size checks and randomized upload names.
- Security headers, restricted CORS and escaped output.
- Database access only through Prisma/parameterized queries.
- No secrets in logs.

Performance targets

Metric	Target
Dashboard/list page load	< 3 seconds
Verification on Sepolia	< 5 seconds
Verification on local chain	< 1 second
Indexer lag on Sepolia	< 30 seconds
List pagination	20 items/page

Animation/performance rules

- Prefer CSS transform/opacity for simple interactions.
- Use lightweight motion for timeline, transaction states and page transitions.
- Avoid continuously animated backgrounds, heavy particle systems and unnecessary 3D/WebGL.
- Disable duplicate actions while a transaction is pending.
- Use skeletons and optimistic UI only where state semantics remain clear.
- Test mobile QR scanning and low-end browser behavior.

Source basis: TRD security, performance and reliability requirements. ■filecite■turn7file7■L524-L545■

16. Phase 12 — Local Demo → Sepolia → Final Deployment

Objective: preserve local reliability while preparing a public testnet version for final demonstration/submission.

Environment strategy

Environment	Blockchain	Database	Purpose
Local	Hardhat node, chain ID 31337	Docker PostgreSQL	Development + reliable demo
Testnet	Sepolia, chain ID 11155111	Supabase or Neon	Final public demo/submission

Deployment sequence

- Freeze MVP code after acceptance criteria pass locally.
- Run complete contract, backend and E2E tests.
- Deploy SupplyChain to Sepolia with a dedicated deployer key stored only in secure environment variables.
- Record contract address, chain ID and deployBlock.
- Configure the hosted PostgreSQL database and run Prisma migrations.
- Start the indexer from deployBlock.
- Seed only safe demo participants/products; never use Hardhat default private keys on Sepolia.
- Configure the web app for Sepolia and verify MetaMask network switching.
- Run the final demo walkthrough end to end.
- Keep the local Hardhat environment documented as a fallback.

Demo seed

The schema design specifies a local seed that creates Admin, Manufacturer, Distributor, Retailer and an extra Distributor for rejection demonstrations, plus five sample products covering Created, InTransit, AtDistributor, AtRetailer and Sold.

Source basis: TRD environments/demo readiness and schema seed design. [filecite turn6file4 L329-L368](#)
[filecite turn7file8 L638-L661](#)

17. MVP Completion Gate

The MVP is complete only when every acceptance criterion below passes on the local chain.

#	Acceptance criterion	Pass condition
1	Admin onboarding	One Manufacturer, Distributor and Retailer are registered
2	Product + QR	Manufacturer registers product and downloads QR
3	Full journey	Manufacturer → Distributor → Retailer → Sold
4	Authorization	Wrong role/owner action is rejected clearly
5	Reject flow	Rejected transfer returns product to sender correctly
6	Public verification	QR without login shows Authentic + ordered timeline
7	Tamper detection	Edited off-chain data produces Data Mismatch
8	Unknown product	Unknown ID produces Not Found
9	Demo reproducibility	Entire flow works locally with one setup guide

Recommended final demo order

- 1. Admin connects wallet and registers the three supply-chain participants.
- 2. Manufacturer creates a product and displays/downloads the QR.
- 3. Manufacturer transfers it to the Distributor.
- 4. Distributor accepts and adds a location update.
- 5. Distributor transfers to Retailer.
- 6. Retailer accepts and marks the product Sold.
- 7. Customer scans the QR and sees Authentic with the full timeline.
- 8. Demonstrate wrong-role rejection.
- 9. Demonstrate transfer rejection.
- 10. Modify off-chain product data and show Data Mismatch.
- 11. Enter an unknown product ID and show Not Found.

Source basis: PRD MVP acceptance criteria. ■filecite■turn6file6■L517-L539■

18. Post-MVP Stretch Roadmap

Stretch features are intentionally deferred until the MVP gate is fully green.

Stretch	Implementation idea	Priority
Batch registration	Create N separate on-chain products, each with unique hash/ID/QR	High
PDF certificate	Generate a printable authenticity/product certificate	Medium
Suspicious scan alert	Track unknown/abnormal verification patterns and flag them	Medium
Recall flag	Add a product/batch recall state or off-chain recall metadata	Medium
Journey map	Plot recorded locations on a map with an animated route	Medium
Analytics	Verification scans, transfer times, status counts and rejection rates	Low
Sender-side cancel	Optional cancellation flow for pending transfers	Low

Do not let stretch work alter the MVP state machine unless the change is explicitly reviewed. The current PRD keeps batch registration, PDF certificates, suspicious alerts, recall, maps and analytics outside the must-ship scope.

Source basis: PRD MVP vs stretch scope and schema consistency notes. ■filecite■turn6file6■L517-L527■
■filecite■turn7file4■L357-L362■

19. Master Implementation Checklist

Use this as the working checklist while building in the AI coding IDE.

Phase	Checklist
0	■ Colors locked ■ Typography ■ Components ■ Screens ■ Animation rules
1	■ Repo ■ Hardhat ■ Next.js ■ PostgreSQL ■ Prisma ■ env ■ scripts
2	■ Roles ■ Product ■ Transfers ■ Reject ■ Location ■ Sold ■ History ■ Events
3	■ Positive tests ■ Negative tests ■ Access tests ■ State machine ■ Coverage
4	■ Schema ■ Migrations ■ API ■ Auth nonce ■ Validation ■ Uploads
5	■ Event listener ■ Resume block ■ Idempotency ■ Snapshot ■ Error handling
6	■ App shell ■ MetaMask ■ Network switch ■ Signature ■ JWT ■ Role routing
7	■ Admin dashboard ■ Add participant ■ Activate/deactivate ■ Activity
8	■ Product form ■ Hash ■ Draft ■ Register ■ Confirm ■ QR
9	■ Manufacturer transfer ■ Distributor accept/reject ■ Location ■ Retailer transfer ■ Sold
10	■ QR scan ■ On-chain read ■ Hash compare ■ Authentic ■ Mismatch ■ Not Found
11	■ Security ■ Performance ■ Responsive ■ Loading ■ Errors ■ E2E
12	■ Local freeze ■ Sepolia deploy ■ DB migration ■ Indexer ■ Final demo ■ Backup

Final principle

Build the blockchain state machine first, then the data layer, then the UI, and only after the end-to-end MVP works should visual polish and stretch features expand. This keeps ChainTrack technically defensible, demonstrable and maintainable.

20. Source Documents & Decision Notes

This implementation plan is grounded in the ChainTrack project documents supplied in the workspace and in the UI/UX decisions finalized during this planning conversation.

Document	Role in this plan
01-PRD.md	Problem, goals, roles, lifecycle, business rules, MVP scope and acceptance criteria
02-TRD.md	Architecture, technology stack, security, performance, testing and deployment
03-System-Flow.md	Detailed flows for login, onboarding, product registration, transfers, verification, indexer and edge cases
04-Schema-and-Contract-Design.md	Smart-contract data model, functions, test cases, database schema, indexer rules and seed design
UI/UX decisions	Navy + Orange light theme, fast purposeful animations, responsive and professional interaction model

Key project decisions locked

- Brand: Navy #173B57 + Orange #F97316 on a light #F8FAFC / white interface.
- Blockchain development: Hardhat local chain first; Sepolia for final public demo.
- Wallet: MetaMask; user transactions are signed by the user's wallet.
- Storage: hybrid; blockchain for integrity/state/history, PostgreSQL for descriptive/search data.
- Customer verification: public QR route with no login.
- MVP first; stretch features only after all acceptance criteria pass.
- Local chain remains the reliable fallback for demonstrations.

End of implementation plan

ChainTrack — Track. Verify. Trust.